



```
def deploy_pipeline():
    project = "ai"
    env = "prod"
    >>> deploy_result =
    agent = CodingAgent(model="gpt-4o")
    >>> agent.execute(project)
    tests = agent.execute_tests()
    if tests.passed:
        return agent.deploy(env)

>>> pipeline = new AIPipeline(
    model="Claude-opus",
    tools=["read", "search", "execute"],
    max_iterations=50
)

>>> pipeline.execute()
Construa um sistema completo
de gestão de pedidos com
testes unitários e deploy
```

A BANCADA DO PROJETO VIVO

PLAYBOOK — A\$ QUATRO CAMADAS DA FÁBRICA AGÊNTICA

◆ PARA INICIANTES

16 capítulos

projetos práticos



AUTOR

Heverton Eduardo Peres

Especialista em Marketing e Desenvolvimento de Soluções

16
CAPÍTULOS

70+
PÁGINAS

∞
PROJETOS

Objetivo do Material

Você não lê este livro: você monta uma bancada. O mesmo projeto real fica sobre a mesa do capítulo 1 ao 16, e cada capítulo instala uma peça que continua no lugar até o fim. Nada de exemplo descartável: o que entra na bancada é usado de novo, todo capítulo, e é isso que faz o aprendizado grudar.

Obra-mãe: tratado-4-camadas-v2

Passos práticos: 16

Você é: Engenheiro de Bancada

Sumário

Objetivo do Material	4
Como usar este playbook	5
Mapa dos Estágios	6
Passos Práticos	7
Passo 1 — A conta que ninguém quer pagar	7
Passo 2 — O dicionário de bancada	8
Passo 3 — O seu projeto na bancada	9
Passo 4 — A Constituição da bancada	11
Passo 5 — Peça 1 — Contexto: o que a IA lê antes de agir	12
Passo 6 — Peça 2 — Harness: o ciclo de vida e os disjuntores	13
Passo 7 — Peça 3 — Motor: roteamento, contratos e custo	15
Passo 8 — Peça 4 — Ferramentas e persistência: a usina determinística	16
Passo 9 — Primeiro encaixe: do script solto ao repositório governado	17
Passo 10 — O caso âncora completo: o Painel de Pedidos, da ideia ao ar	19
Passo 11 — Trabalho em paralelo: subagentes, worktrees e integração sem colisão	20
Passo 12 — Os portões finais: teste, auditoria e entrega	21
Passo 13 — Fazer mais gastando menos: a economia da bancada	23
Passo 14 — O certificado de bancada: provar que funciona	24
Passo 15 — Levando a bancada para o time (e para o código que já existe)	25
Passo 16 — Soberania: não ficar preso a fornecedor, modelo ou plataforma	27
Checklist Mestre	29

Objetivo do Material

Mostrar a conta que o improviso com IA cobra de quem constrói de verdade, apresentar o Painel de Pedidos como caso âncora que atravessa a obra inteira e fixar a promessa concreta: ao final, o leitor tem um projeto real funcionando com as quatro camadas instaladas e evidência de resultado.

Como usar este playbook

Você é o **Engenheiro de Bancada**. Cada passo é um card independente com sete partes: objetivo, pré-requisito, entregas, execução, gate de verificação, critério de conclusão e armadilhas.

Este documento **não repete a teoria** do livro. Quando precisar do porquê, siga a referência cruzada do card para o capítulo correspondente.

Mapa dos Estágios

#	Estágio	Passos
1	Bancada	1, 2, 3, 4
2	Projeto vivo	5, 6, 7, 8
3	Peça instalada	9, 10, 11, 12
4	Encaixe	13, 14, 15, 16

Passos Práticos

Passo 1 — A conta que ninguém quer pagar

Estágio: Bancada · **Origem:** Cap. 1 — A conta que ninguém quer pagar

① Objetivo do passo

Mostrar com números por que projetos construídos no improviso com IA travam, e apresentar o mapa das quatro camadas que resolve cada uma das dores.

② Pré-requisito

Nenhum — este é o ponto de partida

③ Entregas

- inventario-da-conta.py
- caderno-de-bancada.md
- linha-de-base.json

④ Execução

4.1 O inventário da conta

```
TAREFA = "conferencia manual de pedidos"
FREQ_SEMANA, MINUTOS_POR_VEZ = 5, 8

def horas_por_ano(freq, minutos):
    return round(freq * minutos * 52 / 60, 1)

print(TAREFA, "->", horas_por_ano(FREQ_SEMANA, MINUTOS_POR_VEZ), "h/ano")
```

4.3 O caderno de bancada

```
{
  "tarefa": "conferencia manual de pedidos",
  "minutos_por_semana": 40,
  "erros_por_mes": 3,
  "impacto": "pedido conferido errado chega ao cliente"
}
```

⑤ Verificação / Gate

```
python -c "freq, minutos = 5, 8; horas = freq * minutos * 52 / 60; assert horas > 0;
print('linha de base valida:', round(horas, 1), 'h/ano')"
```

⑥ Feito quando...

- ☐ Que tarefa apareceu mais de uma vez nesta semana?
- ☐ Quanto tempo ela levou, contado em blocos de quinze minutos?
- ☐ O resultado dela tem um critério de pronto que outra pessoa conseguiria conferir sem discutir?

⑦ Armadilhas

- Somar trabalho produtivo junto com repetição e concluir que o custo é impossível de resolver.
- Anotar a linha de base sem data — sem data não existe comparação possível no capítulo 16.
- Medir por mês em vez de por tarefa, o que esconde qual repetição está custando caro.

Passo 2 — O dicionário de bancada

Estágio: Bancada · **Origem:** Cap. 2 — O dicionário de bancada

① Objetivo do passo

Fixar sem jargão os termos que aparecem do começo ao fim da obra, cada um com analogia de bancada e um exemplo tirado do caso âncora.

② Pré-requisito

Passo 1 concluído

③ Entregas

- glossario.yaml
- verificar-glossario.py
- termos-do-dominio.md

④ Execução

4.1 O glossário como arquivo do projeto

```
termos:
  lote:
    definicao: conjunto de itens processados juntos
    nao_e: pedido isolado
  fechamento:
    definicao: conferencia que libera um lote
    nao_e: correcao de erro
```

4.2 A verificação de formato

```
TERMOS = {"lote": "conjunto de itens processados juntos",  
          "fechamento": "conferencia que libera um lote"}  
  
for termo, definicao in TERMOS.items():  
    assert len(definicao.split()) <= 12, f"{termo}: definicao longa"  
    print(f"[ok] {termo}")
```

⑤ Verificação / Gate

```
python -c "t={'lote':'conjunto de itens processados juntos'}; assert  
all(len(v.split())<=12 for v in t.values()); print('glossario ok:', len(t), 'termo(s)')"
```

⑥ Feito quando...

- ☐ Situação.** Você assume a continuidade de um projeto de relatórios que outra pessoa começou
- ☐ Existe um arquivo de instruções com um termo que aparece dezenove vezes: “consolidar”
- ☐ Você pede ao agente para “consolidar os pedidos do dia”
- ☐ O erro.** Você culpa o modelo
- ☐ Cada resposta mantém o mesmo termo ambíguo
- ☐ Você apenas adiciona qualificadores em volta dele
- ☐ Ao final do dia

⑦ Armadilhas

- Definir com duas orações subordinadas: se a frase ficou longa, o termo ainda não foi resolvido.
- Deixar a coluna “o que não é” vazia e perder o mecanismo que impedia o sistema de traduzir o problema por conta própria.
- Crescer o glossário sem verificação de formato, o que transforma o arquivo em texto morto.

Passo 3 — O seu projeto na bancada

Estágio: Bancada · **Origem:** Cap. 3 — O seu projeto na bancada

① Objetivo do passo

Preparar o alvo real: escolher o projeto, fotografar o estado atual em números, medir o custo do trabalho manual e criar a base mínima de trabalho.

② Pré-requisito

Passo 2 concluído

③ Entregas

- `cronometro-linha-de-base.py`
- `linha-de-base.json`
- `estrutura-repositorio.md`

④ Execução

4.1 A linha de base em quatro números

```
BASE = {"tempo_de_ciclo_h": 6.0, "retrabalho": 0.2,
        "falhas_semana": 1, "minutos_conferencia": 25}

def custo_de_conferencia(base):
    return round(base["minutos_conferencia"] * 5 / 60, 1)

print("conferencia por semana:", custo_de_conferencia(BASE), "h")
```

4.2 A árvore mínima do repositório

```
pedidos/
  dados/
  verificacoes/
caderno-de-bancada.md
```

⑤ Verificação / Gate

```
python -c "base={'tempo_de_ciclo_h':6.0,'retrabalho':0.2}; assert
set(base)=={'tempo_de_ciclo_h','retrabalho'}; print('linha de base com', len(base),
'numeros dados')"
```

⑥ Feito quando...

- ☐ Situação.** Você decide automatizar a emissão de notas de serviço da sua equipe
- ☐ O erro.** Na terça você descobre que existem duas planilhas: uma da equipe interna
- ☐ No dia 20 você olha para o projeto
- ☐ O diagnóstico.** O erro não foi técnico
- ☐ Sem linha de base
- ☐ É o mesmo padrão que a pesquisa de desempenho de entrega descreve quando a IA é aplicada a um processo que ainda não está definido: a ferramenta amplifica a desorganização existente [1]
- ☐ A correção.** Volte duas casas

⑦ Armadilhas

- Medir as quatro métricas em semanas diferentes e comparar mesmo assim.
- Começar pela taxa de retrabalho, que depende de uma definição estável do que conta como refeito.
- Guardar a linha de base fora do repositório, onde ninguém consegue conferir depois.

Passo 4 — A Constituição da bancada

Estágio: Bancada · **Origem:** Cap. 4 — A Constituição da bancada

① Objetivo do passo

Apresentar as leis que impedem retrabalho e ensinar a escrever as regras do próprio projeto em um arquivo único que a IA lê antes de agir.

② Pré-requisito

Passo 3 concluído

③ Entregas

- `constituicao.yaml`
- `portao-constituicao.py`
- `decisoes.md`

④ Execução

4.1 O arquivo de regras do projeto

```
regras:
  - id: R1
    texto: todo config declara a versao do formato
    verificavel: true
  - id: R2
    texto: nenhum segredo em repositorio
    verificavel: true
```

4.2 O portão que fiscaliza a constituição

```
import re

SEGREGO = re.compile(r"(api[_-]?key|senha|secret)\s*[:=]", re.I)
LINHA = 'token = "abc"'

def fiscalizar(linha):
    return "reprovado" if SEGREGO.search(linha) else "aprovado"

print(fiscalizar(LINHA))
```

⑤ Verificação / Gate

```
python -c "import re; p=re.compile(r'(api[_-]?key|senha)', re.I); assert p.search('senha = x'); print('portao de segredo operante')"
```

⑥ Feito quando...

- ❑ A regra cabe em uma frase afirmativa? ** “Todo arquivo de configuração declara a versão do formato” é uma regra. “Sempre escrever código limpo” é um desejo
- ❑ Existe uma entrada e uma saída observáveis? ** Se você não consegue dizer o que seria reprovado, ninguém consegue construir o portão
- ❑ A reprovação é automática? ** Regra que depende de alguém lembrar de conferir não é portão: é combinado

⑦ Armadilhas

- Escrever regra que ninguém consegue reprovar — sem critério objetivo não existe portão, existe intenção.
- Misturar acordo de time com regra verificável e produzir confiança alta com verificação zero.
- Implementar dez regras de uma vez em vez de uma que roda de verdade.

Passo 5 — Peça 1 — Contexto: o que a IA lê antes de agir

Estágio: Projeto vivo · **Origem:** Cap. 5 — Peça 1 — Contexto: o que a IA lê antes de agir

① Objetivo do passo

Montar a camada de contexto: definir o que entra na mesa de trabalho da IA, em que formato e em que ordem, para que ela obedeça hoje e continue obedecendo depois.

② Pré-requisito

Passo 4 concluído

③ Entregas

- `especificacao-pagina.md`
- `contrato-pedidos.json`
- `exemplo-minimo.csv`

④ Execução

4.1 A especificação de uma tarefa

Tarefa: carregar pedidos do dia
Entrada: pedidos.csv (id, valor, data)
Saída: tabela pedidos com total conferido
Recusa: se faltar id, parar e informar a linha
Limite: não altera pedido já conciliado

4.2 O contrato de dados

```
{
  "campos": ["id", "valor", "data"],
  "tipos": {"id": "texto", "valor": "numero", "data": "data"},
  "obrigatorios": ["id", "valor"]
}
```

⑤ Verificação / Gate

```
python -c "import json; c={'campos':['id','valor'],'obrigatorios':['id']}; assert set(c['obrigatorios'])<=set(c['campos']); print('contrato de contexto coerente')"
```

⑥ Feito quando...

- ☐ Situação.** Você precisa que o agente ajuste a regra de arredondamento do relatório de pedidos
- ☐ Para “dar contexto”
- ☐ O pedido fica com alguns milhares de linhas
- ☐ O erro.** A resposta chega bonita
- ☐ O diagnóstico.** A causa é a mesa
- ☐ Você misturou quatro coisas de naturezas diferentes: instrução (a regra de arredondamento)
- ☐ Quando tudo tem o mesmo peso

⑦ Armadilhas

- Descrever passos em vez de resultado e transformar a especificação em manual frágil.
- Deixar o comportamento de recusa em silêncio, que é o que produz resposta inventada com aparência plausível.
- Encher o contexto de arquivo irrelevante e deixar fora justamente o dado que faltava.

Passo 6 — Peça 2 — Harness: o ciclo de vida e os disjuntores

Estágio: Projeto vivo · **Origem:** Cap. 6 — Peça 2 — Harness: o ciclo de vida e os disjuntores

① Objetivo do passo

Instalar o ciclo de vida do trabalho com portões que devolvem sim ou não e proteções que impedem ações destrutivas antes que elas aconteçam.

② Pré-requisito

Passo 5 concluído

③ Entregas

- portoes.yaml

- executar-portoes.py
- disjuntor-escrita.py

④ Execução

4.1 O manifesto de portões

```
portoes:
  - nome: sintaxe
    comando: python -m py_compile app.py
  - nome: teste
    comando: python -m pytest -q
disjuntores:
  tentativas: 3
  escopo: ["app.py", "testes/"]
```

4.2 O disjuntor de tentativas

```
LIMITE = 3

def executar(passos):
    for i, passo in enumerate(passos, 1):
        if i > LIMITE:
            return f"parado no passo {i}: limite de tentativas"
        print("passo", i, passo)
    return "concluido"

print(executar(["a", "b", "c", "d"]))
```

⑤ Verificação / Gate

```
python -c "limite=3; passos=4; assert passos>limite; print('disjuntor de tentativas armado em', limite)"
```

⑥ Feito quando...

- ☐ Situação.** É sexta-feira
- ☐ Você tem pressa
- ☐ Não roda a verificação porque ela demora quarenta segundos
- ☐ O erro.** O agente interpreta “ajustar” como reorganizar
- ☐ Resultado: o arquivo do dia foi importado duas vezes
- ☐ O diagnóstico.** Não faltou cuidado: faltaram disjuntor
- ☐ O disjuntor de escrita teria bloqueado a gravação na pasta de origem

⑦ Armadilhas

- Disjuntor que só registra aviso: sem ação de parada ele é log, não proteção.
- Calibrar o limite de tentativas pelo valor do executor em vez do custo da conferência.
- Deixar a escrita fora de escopo declarado e permitir alteração em arquivo que ninguém pediu.

Passo 7 — Peça 3 — Motor: roteamento, contratos e custo

Estágio: Projeto vivo · **Origem:** Cap. 7 — Peça 3 — Motor: roteamento, contratos e custo

① Objetivo do passo

Decidir quem executa cada tarefa, travar as respostas em formatos previsíveis e entender como o custo cresce junto com o contexto.

② Pré-requisito

Passo 6 concluído

③ Entregas

- roteamento.yaml
- contrato-resposta.py
- custo-por-tarefa.py

④ Execução

4.1 A tabela de decisão de roteamento

```
portas:
  deterministica: script
  regra_fixa: portao
  ambigua: rota_de_julgamento
  limite_custo_por_tarefa: 0.50
```

4.2 O contrato de resposta e o validador

```
CONTRATO = {"campos": ["decisao", "evidencia"]}

def validar(resposta):
    faltando = [c for c in CONTRATO["campos"] if c not in resposta]
    if faltando:
        return f"recusado: falta {faltando}"
    return "aprovado"

print(validar({"decisao": "usar script"}))
print(validar({"decisao": "usar script", "evidencia": "teste 12"}))
```

⑤ Verificação / Gate

```
python -c "c={'decisao':1,'evidencia':2}; assert {'decisao','evidencia'}<=set(c);
print('contrato de resposta completo')"
```

⑥ Feito quando...

- ☐ Situação.** Você recebe a tarefa de transformar os totais do dia em uma tabela de cinco linhas para colar no grupo do time
- ☐ Cansado de fazer à mão
- ☐ O erro.** A resposta vem com sete linhas
- ☐ Você ajusta o pedido
- ☐ Depois de quatro tentativas
- ☐ O diagnóstico.** A tarefa é determinística: os cinco totais já existem no arquivo de saída
- ☐ Não havia ambiguidade a resolver nem caminho a descobrir — portanto

⑦ Armadilhas

- Rodar tarefa determinística na rota de julgamento e pagar duas vezes: chamada e revisão.
- Trocar por modelo mais barato sem medir a taxa de reprovação depois da troca.
- Aceitar saída sem contrato declarado, o que torna a verificação impossível de automatizar.

Passo 8 — Peça 4 — Ferramentas e persistência: a usina determinística

Estágio: Projeto vivo · **Origem:** Cap. 8 — Peça 4 — Ferramentas e persistência: a usina determinística

① Objetivo do passo

Montar as mãos da operação: ferramentas reexecutáveis, padronização de conexão com o mundo externo e um registro durável do que foi feito.

② Pré-requisito

Passo 7 concluído

③ Entregas

- `ferramentas.yaml`
- `importar-idempotente.py`
- `registro-execucao.py`

④ Execução

4.1 A etiqueta das ferramentas

```
ferramenta: importar_pedidos
escopo: [ler, gravar]
idempotente: true
chave_unica: id_externo
```

4.2 A importação idempotente

```
REGISTRO = []

def importar(valor, chave):
    if any(r["chave"] == chave for r in REGISTRO):
        return "pulado: ja importado"
    REGISTRO.append({"chave": chave, "valor": valor})
    return "importado"

print(importar(10, "P-1"))
print(importar(10, "P-1"))
print("registros:", len(REGISTRO))
```

⑤ Verificação / Gate

```
python -c "vistos=set(); chave='P-1'; vistos.add(chave); assert len(vistos)==1;
print('escrita idempotente por chave unica')"
```

⑥ Feito quando...

- ☐ Aceita entrada diferente? Se só funciona com o arquivo daquele dia, ainda é script pessoal
- ☐ Sai com código de erro claro? Falha silenciosa obriga a pessoa a investigar o que aconteceu
- ☐ Pode rodar duas vezes sem estragar? Operação que não é segura para repetir transfere risco para quem aperta o botão [8]
- ☐ Deixa rastro do que fez? Sem registro, a conferência volta a depender de memória

⑦ Armadilhas

- Ferramenta que só funciona com o arquivo daquele dia — ainda é script pessoal.
- Rodar duas vezes e duplicar dado, o que transfere risco para quem aperta o botão.
- Falhar sem código de erro claro e obrigar a pessoa a investigar o que aconteceu.

Passo 9 — Primeiro encaixe: do script solto ao repositório governado

Estágio: Peça instalada · **Origem:** Cap. 9 — Primeiro encaixe: do script solto ao repositório governado

① Objetivo do passo

Juntar as quatro peças em um projeto pequeno e real, partindo do que já existe, e fechar o primeiro ciclo completo com aprovação.

② Pré-requisito

Passo 8 concluído

③ Entregas

- inventario-projeto.py
- roteiro-instalacao.md
- registro-ciclo.json

④ Execução

4.2 O roteiro de instalação das quatro peças

```
$ git init && git add . && git commit -m "estado inicial"
$ python verificacoes/medir_contexto.py
[ok] contexto com 4 arquivos
```

4.4 O registro do primeiro ciclo

```
{
  "tarefa": "conferir totais do relatorio",
  "entrada": "dados/pedidos.csv",
  "portao": "python verificacoes/medir_contexto.py",
  "resultado": "aprovado"
}
```

⑤ Verificação / Gate

```
python -c "import json; c={'tarefa':'conferir totais','resultado':'aprovado'}; assert
'resultado' in c; print('primeiro ciclo registrado')"
```

⑥ Feito quando...

- ☐ Situação.** Você assume um projeto que gera relatórios de produção: um script de 180 linhas
- ☐ A primeira decisão que vem à cabeça é reescrever o script inteiro com apoio de IA
- ☐ O erro.** Você reescreve
- ☐ A versão nova é mais bonita
- ☐ Na primeira madrugada ela roda
- ☐ Ninguém sabia disso
- ☐ O diagnóstico.** Você reescreveu antes de inventariar

⑦ Armadilhas

- Escolher a tarefa mais interessante em vez da mais chata: a chata tem critério de pronto mais claro.
- Parar na etapa 3 e culpar o executor quando o problema era o contexto.
- Não manter o caso do primeiro encaixe como referência para testar as peças seguintes.

Passo 10 — O caso âncora completo: o Painel de Pedidos, da ideia ao ar

Estágio: Peça instalada · **Origem:** Cap. 10 — O caso âncora completo: o Painel de Pedidos, da ideia ao ar

① Objetivo do passo

Percorrer a construção completa do caso âncora, com as decisões, os erros e as correções que aparecem em um projeto de verdade.

② Pré-requisito

Passo 9 concluído

③ Entregas

- `esquema.sql`
- `importar-com-conferencia.py`
- `relatorio-totais.py`

④ Execução

4.1 O esquema do banco

```
CREATE TABLE pedidos (  
  id_externo TEXT PRIMARY KEY,  
  valor REAL NOT NULL,  
  data TEXT NOT NULL  
);
```

4.3 O relatório com portão de totais

```
PEDIDOS = [{"id": "P-1", "valor": 30.0}, {"id": "P-2", "valor": 12.5}]  
  
def conferir(linhas, total_manual):  
    total = round(sum(p["valor"] for p in linhas), 2)  
    return {"total": total, "fecha": abs(total - total_manual) < 0.01}  
  
print(conferir(PEDIDOS, 42.5))
```

⑤ Verificação / Gate

```
python -c "linhas=[{'v':30.0},{ 'v':12.5}]; total=round(sum(l['v'] for l in linhas),2);  
assert total==42.5; print('totais fecham:', total)"
```

⑥ Feito quando...

- ☐ Declarar o problema em uma frase, com quem sofre e o que muda
- ☐ Registrar a linha de base antes de escrever qualquer linha de código
- ☐ Escrever o glossário mínimo dos termos do domínio
- ☐ Definir a primeira regra verificável do projeto
- ☐ Especificar a tarefa inicial com entrada, saída e recusa
- ☐ Montar o contexto com os arquivos que realmente importam
- ☐ Rodar o ciclo com disjuntor e portão ligados

⑦ Armadilhas

- Publicar sem medir uma semana de uso: versão que ninguém usa não gera aprendizado.
- Ampliar escopo antes de instrumentar, e descobrir o erro pelo relato do usuário.
- Deixar o identificador externo de fora, o que torna a duplicidade indetectável.

Passo 11 — Trabalho em paralelo: subagentes, worktrees e integração sem colisão

Estágio: Peça instalada · **Origem:** Cap. 11 — Trabalho em paralelo: subagentes, worktrees e integração sem colisão

① Objetivo do passo

Dividir trabalho entre vários agentes sem que um apague o outro, com isolamento físico, fila de tarefas, revisão e integração controlada.

② Pré-requisito

Passo 10 concluído

③ Entregas

- plano-frentes.yaml
- conferir-escopo.py
- fila-estados.py

④ Execução

4.2 Criar e descartar frentes isoladas

```
$ git worktree add ../pedidos-relatorio -b frente/relatorio
$ git worktree list
$ git worktree remove ../pedidos-relatorio
```

4.4 Conferência de escopo antes de integrar

```
ARQUIVOS_A = {"relatorio.py"}
ARQUIVOS_B = {"importacao.py"}

def pode_paralelizar(a, b):
    return not (a & b)

print(pode_paralelizar(ARQUIVOS_A, ARQUIVOS_B))
print(pode_paralelizar(ARQUIVOS_A, {"relatorio.py", "app.py"}))
```

⑤ Verificação / Gate

```
python -c "a={'x'}; b={'y'}; assert not (a & b); print('frentes sem colisao de arquivos')"
```

⑥ Feito quando...

- ☐ Situação.** Você descobre o paralelismo
- ☐ Em meia hora
- ☐ O erro.** No fim do dia
- ☐ Metade do trabalho é descartada
- ☐ O diagnóstico.** Três falhas simultâneas: sem isolamento físico
- ☐ O custo não foi de execução
- ☐ Somando a isso a sobreposição de escopo

⑦ Armadilhas

- Paralelizar dois trabalhos que escrevem nos mesmos arquivos e pagar mais na integração do que ganhou no tempo.
- Passar de duas frentes e transformar a atenção do operador em gargalo.
- Integrar sem verificação final só porque cada parte passou isolada.

Passo 12 — Os portões finais: teste, auditoria e entrega

Estágio: Peça instalada · **Origem:** Cap. 12 — Os portões finais: teste, auditoria e entrega

① Objetivo do passo

Fechar o trabalho com verificação séria: testes que provam comportamento, auditoria que aponta o que ficou frouxo e uma entrega que outra pessoa consegue usar.

② Pré-requisito

Passo 11 concluído

③ Entregas

- testes-comportamento.py
- auditoria-evidencia.py
- pacote-entrega.md

④ Execução

4.2 O teste que falha antes e passa depois

```
def somar_total(pedidos):  
    return round(sum(p["valor"] for p in pedidos), 2)  
  
def teste_falha_antes():  
    pedidos = [{"valor": 10.0}, {"valor": 5.5}]  
    assert somar_total(pedidos) == 15.5, "total divergente"  
    return "passou"  
  
print(teste_falha_antes())
```

4.3 A auditoria por evidência

Afirmacoes com evidencia:

- total fecha -> teste_equivalencia.py

Limites declarados:

- nao cobre pedido cancelado

⑤ Verificação / Gate

```
python -c "f=lambda p: round(sum(x['valor'] for x in p),2); assert f([{'valor':10.0}, {'valor':5.5}])==15.5; print('teste verde')"
```

⑥ Feito quando...

- ☐ Escreva o teste no estado atual.** Ele deve falhar, porque a correção ainda não existe
- ☐ Anote a mensagem de falha.** Ela é a prova de que o teste mede a coisa certa e não passa por acidente
- ☐ Aplique a correção mínima.** Sem melhorias de carona junto — misturar as duas coisas impede saber o que corrigiu
- ☐ Rode a suíte inteira.** O teste novo passa e nenhum antigo quebrou

⑦ Armadilhas

- Escrever o teste depois do código, o que não prova que ele reproduzia o defeito.
- Misturar refatoração com correção no mesmo passo e perder a atribuição do que resolveu.
- Confundir teste automatizado com auditoria de entrega: um cobre o previsto, o outro o imprevisto.

Passo 13 — Fazer mais gastando menos: a economia da bancada

Estágio: Encaixe · **Origem:** Cap. 13 — Fazer mais gastando menos: a economia da bancada

① Objetivo do passo

Medir e reduzir o custo da operação sem perder qualidade, usando contexto reaproveitado, tarefas no lugar certo e limites claros de uso.

② Pré-requisito

Passo 12 concluído

③ Entregas

- custo-por-tarefa.py
- corte-de-rota.py
- prefixo-estavel.yaml

④ Execução

4.1 O registro de custo por tarefa

```
REGISTRO = [  
    {"tarefa": "conferir totais", "rota": "agente", "custo": 0.42, "tentativas": 3},  
    {"tarefa": "renomear arquivos", "rota": "script", "custo": 0.0, "tentativas": 1},  
]  
  
def mais_cara(registro):  
    return max(registro, key=lambda r: r["custo"])  
  
print(mais_cara(REGISTRO))
```

4.3 A configuração do prefixo estável

```
prefixo_estavel:  
- constituicao.yaml  
- glossario.yaml  
- contrato-pedidos.json
```

⑤ Verificação / Gate

```
python -c "r=[{'custo':0.42},{ 'custo':0.0}]; assert max(r,key=lambda x:x['custo'])  
['custo']==0.42; print('tarefa mais cara identificada')"
```

⑥ Feito quando...

- ☐ Situação.** Chega a fatura do mês
- ☐ Sua primeira reação é trocar tudo pelo modelo mais barato disponível
- ☐ O erro.** Você troca o modelo
- ☐ O relatório semanal passa a exigir duas correções manuais
- ☐ No mês seguinte
- ☐ O diagnóstico.** Você cortou a alavanca errada
- ☐ O custo estava concentrado em retrabalho

⑦ Armadilhas

- Cortar o preço unitário sem olhar retrabalho e trocar um custo visível por um invisível.
- Aprovar vários cortes no mesmo dia e não saber qual deles causou o efeito.
- Reduzir verificação para caber no orçamento, transferindo o custo para quem usa.

Passo 14 — O certificado de bancada: provar que funciona

Estágio: Encaixe · **Origem:** Cap. 14 — O certificado de bancada: provar que funciona

① Objetivo do passo

Construir evidência de confiabilidade, distinguir prova de opinião e escrever um certificado honesto sobre o que a solução faz e o que não faz.

② Pré-requisito

Passo 13 concluído

③ Entregas

- certificado-bancada.md
- teste-reprodutibilidade.py
- conferencia-amostra.py

④ Execução

4.1 O certificado

```
# Certificado de bancada
Versao do artefato: 1.2
Amostra: 3 de 12 afirmacoes reabertas
Limites: nao cobre pedido cancelado
```

4.3 A conferência por amostra

```
AFIRMACOES = [
    {"texto": "totais fecham", "fonte": "teste_equivalencia.py"},
    {"texto": "sem segredo no repo", "fonte": "portao_constituicao.py"},
]
```

```

]

def conferir(amostra):
    return [a["fonte"] for a in amostra]

print(conferir(AFIRMACOES[:2]))

```

⑤ Verificação / Gate

```

python -c "a=[{'fonte':'t.py'}]; assert all('fonte' in x for x in a); print('afirmacoes com fonte rastreavel')"

```

⑥ Feito quando...

- ☐ Reproduza o procedimento.** Rode como está escrito, sem consultar quem fez
- ☐ Confira três números por amostra.** Escolha três afirmações e vá até a fonte de cada uma
- ☐ Procure o limite declarado.** Se a seção de limites estiver vazia, a entrega reprova
- ☐ Compare com a linha de base.** Sem comparação, o número não significa nada
- ☐ Assine o resultado.** Quem conferiu, quando e com qual versão do artefato

⑦ Armadilhas

- Certificado sem seção de limites: o que não foi verificado precisa estar escrito.
- Depender de quem fez a entrega para reproduzir o procedimento e chamar isso de verificação.
- Exibir amostra como se fosse censo e transformar conferência parcial em promessa total.

Passo 15 — Levando a bancada para o time (e para o código que já existe)

Estágio: Encaixe • **Origem:** Cap. 15 — Levando a bancada para o time (e para o código que já existe)

① Objetivo do passo

Adotar a bancada em equipe e em sistemas herdados: por onde começar, como convencer com evidência e como conviver com código antigo.

② Pré-requisito

Passo 14 concluído

③ Entregas

- plano-adocao.yaml
- painel-adocao.py
- cerca-modulo.md

④ Execução

4.1 O plano de adoção

```
tarefa: conferir totais do relatorio semanal
responsavel: ana
portao: python verificacoes/portoes.py
semana_1: especificacao e contexto
semana_2: execucao com portao ligado
```

4.3 O painel de adoção

```
EXECUCOES = [{"quem": "ana", "rodou": True}, {"quem": "bruno", "rodou": False}]

def adocao(registros):
    total = len(registros)
    usaram = sum(1 for r in registros if r["rodou"])
    return f"{usaram}/{total} com uso observado"

print(adocao(EXECUCOES))
```

⑤ Verificação / Gate

```
python -c "r=[{'rodou':True}]; assert sum(1 for x in r if x['rodou'])==1; print('adocao medida por uso observado')"
```

⑥ Feito quando...

- ☐ Situação.** Você decide apresentar a bancada na reunião semanal
- ☐ Prepara quarenta slides com as quatro camadas
- ☐ A recepção é educada
- ☐ Na semana seguinte
- ☐ O erro.** Você insiste: monta um repositório-modelo completo
- ☐ Duas semanas depois
- ☐ A outra metade abandonou

⑦ Armadilhas

- Anunciar adoção sem dono nomeado: iniciativa compartilhada sem responsável não acontece.
- Refatorar módulo herdado antes de descrever o comportamento atual e perder a referência do que era defeito antigo.
- Medir adoção por declaração de intenção em vez de uso observado.

Passo 16 — Soberania: não ficar preso a fornecedor, modelo ou plataforma

Estágio: Encaixe · **Origem:** Cap. 16 — Soberania: não ficar preso a fornecedor, modelo ou plataforma

① Objetivo do passo

Proteger o que foi construído, garantindo que dados, regras e histórico continuem seus e que a troca de fornecedor seja uma decisão e não uma emergência.

② Pré-requisito

Passo 15 concluído

③ Entregas

- contrato-portabilidade.yaml
- teste-portabilidade.py
- decisoes-risco.md

④ Execução

4.1 O contrato de portabilidade

```
chamada: rota_de_julgamento
formato_entrada: json
formato_saida: json
troca_permitida: true
custo_da_troca: 2 h
```

4.2 O teste de portabilidade

```
def mesma_saida(entrada):
    a = {"ok": entrada["valor"] > 0, "custo": 0.010}
    b = {"ok": entrada["valor"] > 0, "custo": 0.008}
    return {"aprovacao_igual": a["ok"] == b["ok"],
            "custo_a": a["custo"], "custo_b": b["custo"]}

print(mesma_saida({"valor": 10}))
```

⑤ Verificação / Gate

```
python -c "f=lambda v: {'ok': v>0}; assert f(10)['ok']==f(10)['ok']; print('saida equivalente entre recursos')"
```

⑥ Feito quando...

- ☐ Escolha uma tarefa de rotina.** Não a mais crítica, nem a mais fácil: uma que roda toda semana
- ☐ Congele a especificação e o contexto.** Eles não mudam durante o teste
- ☐ Rode no recurso novo.** Mesma entrada, mesmo critério de pronto
- ☐ Compare três números.** Taxa de aprovação no portão, custo por execução e tempo até o resultado
- ☐ Registre o veredito.** Fica, volta ou vale como plano B declarado

⑦ Armadilhas

- Trocar de fornecedor sem congelar especificação e contexto, e medir outra tarefa por engano.
- Manter duas rotas ativas em produção e dobrar a superfície de manutenção e verificação.
- Decidir por preço do momento e não revisar a decisão quando o cenário de modelos muda.

Checklist Mestre

Passo 1 — A conta que ninguém quer pagar

- ☐ Que tarefa apareceu mais de uma vez nesta semana?
- ☐ Quanto tempo ela levou, contado em blocos de quinze minutos?
- ☐ O resultado dela tem um critério de pronto que outra pessoa conseguiria conferir sem discutir?

Passo 2 — O dicionário de bancada

- ☐ Situação.** Você assume a continuidade de um projeto de relatórios que outra pessoa começou
- ☐ Existe um arquivo de instruções com um termo que aparece dezenove vezes: “consolidar”
- ☐ Você pede ao agente para “consolidar os pedidos do dia”
- ☐ O erro.** Você culpa o modelo
- ☐ Cada resposta mantém o mesmo termo ambíguo
- ☐ Você apenas adiciona qualificadores em volta dele
- ☐ Ao final do dia

Passo 3 — O seu projeto na bancada

- ☐ Situação.** Você decide automatizar a emissão de notas de serviço da sua equipe
- ☐ O erro.** Na terça você descobre que existem duas planilhas: uma da equipe interna
- ☐ No dia 20 você olha para o projeto
- ☐ O diagnóstico.** O erro não foi técnico
- ☐ Sem linha de base
- ☐ É o mesmo padrão que a pesquisa de desempenho de entrega descreve quando a IA é aplicada a um processo que ainda não está definido: a ferramenta amplifica a desorganização existente [1]
- ☐ A correção.** Volte duas casas

Passo 4 — A Constituição da bancada

- ☐ A regra cabe em uma frase afirmativa?** “Todo arquivo de configuração declara a versão do formato” é uma regra. “Sempre escrever código limpo” é um desejo
- ☐ Existe uma entrada e uma saída observáveis?** Se você não consegue dizer o que seria reprovado, ninguém consegue construir o portão
- ☐ A reprovação é automática?** Regra que depende de alguém lembrar de conferir não é portão: é combinado

Passo 5 — Peça 1 — Contexto: o que a IA lê antes de agir

- ☐ Situação.** Você precisa que o agente ajuste a regra de arredondamento do relatório de pedidos
- ☐ Para “dar contexto”
- ☐ O pedido fica com alguns milhares de linhas
- ☐ O erro.** A resposta chega bonita
- ☐ O diagnóstico.** A causa é a mesa
- ☐ Você misturou quatro coisas de naturezas diferentes: instrução (a regra de arredondamento)
- ☐ Quando tudo tem o mesmo peso

Passo 6 — Peça 2 — Harness: o ciclo de vida e os disjuntores

- ☐ Situação.** É sexta-feira
- ☐ Você tem pressa
- ☐ Não roda a verificação porque ela demora quarenta segundos
- ☐ O erro.** O agente interpreta “ajustar” como reorganizar
- ☐ Resultado: o arquivo do dia foi importado duas vezes
- ☐ O diagnóstico.** Não faltou cuidado: faltaram disjuntor
- ☐ O disjuntor de escrita teria bloqueado a gravação na pasta de origem

Passo 7 — Peça 3 — Motor: roteamento, contratos e custo

- ☐ Situação.** Você recebe a tarefa de transformar os totais do dia em uma tabela de cinco linhas para colar no grupo do time
- ☐ Cansado de fazer à mão
- ☐ O erro.** A resposta vem com sete linhas
- ☐ Você ajusta o pedido
- ☐ Depois de quatro tentativas
- ☐ O diagnóstico.** A tarefa é determinística: os cinco totais já existem no arquivo de saída
- ☐ Não havia ambiguidade a resolver nem caminho a descobrir — portanto

Passo 8 — Peça 4 — Ferramentas e persistência: a usina determinística

- ☐ Aceita entrada diferente? Se só funciona com o arquivo daquele dia, ainda é script pessoal
- ☐ Sai com código de erro claro? Falha silenciosa obriga a pessoa a investigar o que aconteceu
- ☐ Pode rodar duas vezes sem estragar? Operação que não é segura para repetir transfere risco para quem aperta o botão [8]
- ☐ Deixa rastro do que fez? Sem registro, a conferência volta a depender de memória

Passo 9 — Primeiro encaixe: do script solto ao repositório governado

- ☐ Situação.** Você assume um projeto que gera relatórios de produção: um script de 180 linhas
- ☐ A primeira decisão que vem à cabeça é reescrever o script inteiro com apoio de IA
- ☐ O erro.** Você reescreve
- ☐ A versão nova é mais bonita
- ☐ Na primeira madrugada ela roda
- ☐ Ninguém sabia disso
- ☐ O diagnóstico.** Você reescreveu antes de inventariar

Passo 10 — O caso âncora completo: o Painele de Pedidos, da ideia ao ar

- ☐ Declarar o problema em uma frase, com quem sofre e o que muda
- ☐ Registrar a linha de base antes de escrever qualquer linha de código
- ☐ Escrever o glossário mínimo dos termos do domínio
- ☐ Definir a primeira regra verificável do projeto
- ☐ Especificar a tarefa inicial com entrada, saída e recusa
- ☐ Montar o contexto com os arquivos que realmente importam
- ☐ Rodar o ciclo com disjuntor e portão ligados

Passo 11 — Trabalho em paralelo: subagentes, worktrees e integração sem colisão

- ☐ Situação.** Você descobre o paralelismo
- ☐ Em meia hora

- ☐ O erro.** No fim do dia
- ☐ Metade do trabalho é descartada
- ☐ O diagnóstico.** Três falhas simultâneas: sem isolamento físico
- ☐ O custo não foi de execução
- ☐ Somando a isso a sobreposição de escopo

Passo 12 — Os portões finais: teste, auditoria e entrega

- ☐ Escreva o teste no estado atual.** Ele deve falhar, porque a correção ainda não existe
- ☐ Anote a mensagem de falha.** Ela é a prova de que o teste mede a coisa certa e não passa por acidente
- ☐ Aplique a correção mínima.** Sem melhorias de carona junto — misturar as duas coisas impede saber o que corrigiu
- ☐ Rode a suíte inteira.** O teste novo passa e nenhum antigo quebrou

Passo 13 — Fazer mais gastando menos: a economia da bancada

- ☐ Situação.** Chega a fatura do mês
- ☐ Sua primeira reação é trocar tudo pelo modelo mais barato disponível
- ☐ O erro.** Você troca o modelo
- ☐ O relatório semanal passa a exigir duas correções manuais
- ☐ No mês seguinte
- ☐ O diagnóstico.** Você cortou a alavanca errada
- ☐ O custo estava concentrado em retrabalho

Passo 14 — O certificado de bancada: provar que funciona

- ☐ Reproduza o procedimento.** Rode como está escrito, sem consultar quem fez
- ☐ Confira três números por amostra.** Escolha três afirmações e vá até a fonte de cada uma
- ☐ Procure o limite declarado.** Se a seção de limites estiver vazia, a entrega reprova
- ☐ Compare com a linha de base.** Sem comparação, o número não significa nada
- ☐ Assine o resultado.** Quem conferiu, quando e com qual versão do artefato

Passo 15 — Levando a bancada para o time (e para o código que já existe)

- ☐ Situação.** Você decide apresentar a bancada na reunião semanal
- ☐ Prepara quarenta slides com as quatro camadas
- ☐ A recepção é educada
- ☐ Na semana seguinte
- ☐ O erro.** Você insiste: monta um repositório-modelo completo
- ☐ Duas semanas depois
- ☐ A outra metade abandonou

Passo 16 — Soberania: não ficar preso a fornecedor, modelo ou plataforma

- ☐ Escolha uma tarefa de rotina.** Não a mais crítica, nem a mais fácil: uma que roda toda semana
- ☐ Congele a especificação e o contexto.** Eles não mudam durante o teste
- ☐ Rode no recurso novo.** Mesma entrada, mesmo critério de pronto
- ☐ Compare três números.** Taxa de aprovação no portão, custo por execução e tempo até o resultado
- ☐ Registre o veredito.** Fica, volta ou vale como plano B declarado